@VANZA

UT5. La Estructura de Código en Python

CFGS

Desarrollo de Aplicaciones Multiplataforma

Dpto. Informática
IES AUGUSTÓBRIGA (NAVALMORAL DE LA MATA)

Contenido

1. Introducción. 2
2. Diferentes componentes de un módulo. 2
3. Estructura básica de los paquetes. 3
3.1. Importación de código Python....3
3.2. Importar contenido en *__init__.py* 6
3.3. Repositorios de paquetes.....7
4. Paquetes disponibles en la librería estándar. 9

1. Introducción.

Al igual que ocurre cuando escribimos un texto, se puede establecer una jerarquía de agrupaciones clara, por ejemplo, los caracteres se agrupan en palabras, las palabras en frases, las frases en párrafos…, en Python se pueden categorizar las partes de un desarrollo o un proyecto:

- **Sentencias**: son la unidad mínima. Asignaciones, ecuaciones, expresiones, comandos, etc. Normalmente no ocupan más de una línea.
- **Bloques de código**: son agrupaciones de sentencias con una lógica específica.
- **Funciones**: compuestas de uno o varios bloques de código.
- **Clases**: definen lógica y se suelen componer de atributos y métodos que guardan información y lógica.
- **Módulos**: recogen funciones o clases y se guardan en un fichero con extensión **.py**
- **Paquetes o librerías**: recogen uno o muchos módulos con una lógica general.
- **Frameworks**: suelen englobar muchos paquetes que forman parte de un todo.

2. Diferentes componentes de un módulo.

Una peculiaridad de los módulos es que pueden ser ejecutados por el intérprete de Python si definen la sentencia:

if __name__ == '__main__' :

Esto ocurre porque cuando el intérprete ejecuta un módulo usando **python <nombre_modulo.py>,** por ejemplo, en ese módulo la variable __name__ se asigna con el valor '__main__' de manera que la expresión condicional del if anterior se cumple.

Pero esto no significa que deba ser obligatoria la existencia de esta sentencia para que un módulo pueda ser ejecutado por el intérprete. Lo que sí está claro es que su uso en muy aconsejable si queremos evitar que el código dentro del módulo se ejecute automáticamente cuando el módulo es importado por otro script:

Ejemplo:

Archivo: **modulo.py**

```
def saludar():
    print("Hola Mundo!");
saludar()
```

Si guardamos el módulo anterior y lo ejecutamos, mostrará "Hola Mundo".

Pues bien, si ahora ese módulo lo importamos en un nuevo módulo y ejecutamos ese módulo veremos que la función **saludar()** se ejecuta automáticamente mostrando el "Hola Mundo" independientemente de lo que haga el resto del nuevo módulo:

Archivo: **nuevoMódulo.py**

```
import modulo # se ejecutará automáticamente
def nuevoSaludo():
    print("Hola desde el nuevo módulo")
nuevoSaludo()
```

Prueba a añadir las siguientes sentencias en el módulo **modulo.py** para comprobar cómo se evita que se ejecute automáticamente cuando se importa:

```
if __name__ == "__main__":
    saludar()
```

3. Estructura básica de los paquetes.

Un paquete es la forma de encapsular múltiples módulos con un nombre en común.

Básicamente es un directorio que contiene módulos y un archivo especial llamado **__init__.py.** Por lo general es un fichero vacío, pero se ejecuta y se lee por el importador de Python cuando se requiere acceder a cualquier contenido dentro del paquete. Por este motivo, si se define cualquier instrucción en el fichero, se ejecutará antes de importar cualquier información.

3.1. Importación de código Python

Los módulos deben ser accesibles desde el intérprete que los pretenda buscar por lo que es necesario que esté definida la ruta (o rutas) en la variable de entorno PYTHONPATH.

Para poder realizar una importación se utiliza el comando **import**. Normalmente la importación se hace de forma **absoluta** de manera que el punto raíz se establece desde la ruta definida en la variable PYTHONPATH o desde la carpeta donde se esté ejecutando el intérprete.

Imagina que tenemos la siguiente estructura en nuestro proyecto:

```
mi_proyecto/
├── main.py
├── mi_paquete/
│   ├── __init__.py   # Indica que es un paquete
│   ├── modulo1.py
│   ├── modulo2.py
│   └── subpaquete/
│       ├── __init__.py
│       └── modulo3.py
```

Suponemos que dentro del modulo1.py tenemos definida una función mi_funcion()

Suponemos que dentro del modulo3.py tenemos definida una función otra_funcion()

Desde main.py podemos importar de distintas formas:

- **import mi_paquete.modulo1:** para usar cualquier recurso (**mi_funcion()**) debemos llamar **mi_paquete.modulo1.mi_funcion()**

- **from mi_paquete.modulo1 import mi_funcion():** para usar el recurso sólo tendré que llamarlo **mi_funcion()**

- **from mi_paquete.subpaquete.modulo3 import otra_funcion():** ahora sólo tendremos que llamar al recurso **otra_funcion()**

- **import mi_paquete.modulo1 as mod1**

- **from mi_paquete.modulo1 import mi_funcion() as mf**

  Las dos formas anteriores nos permiten importar definiendo alias para el recurso o el módulo completo que puede ser muy útil en determinadas ocasiones como por ejemplo cuando haya nomenclaturas comúnmente utilizadas.

- **from mi_paquete.modulo1 import *:** importa todo lo que está dentro de modulo1. No es muy aconsejable.

Un ejemplo sencillo para ver las distintas formas de importar.

Tenemos la siguiente estructura:

```
mi_proyecto/
├── main.py
├── utils/
│   ├── __init__.py
│   ├── calculadora.py
│   ├── texto.py
```

calculadora.py

```python
def suma(a, b):
    return a + b

def resta(a, b):
    return a - b

def multiplicacion(a, b):
    return a * b

def division(a, b):
    if b == 0:
        return "No se puede dividir por 0"
    return a / b

def potencia(a, b):
    return a ** b
```

texto.py

```python
def mayusculas(cadena):
    return cadena.upper()

def minusculas(cadena):
    return cadena.lower()
```

Vemos las diferentes formas de importar en el módulo **main.py**

```
#  Importar el módulo completo
import utils.calculadora

resulSuma = utils.calculadora.suma(5, 3)
print("Suma:", resulSuma)
resulMultiplicacion = utils.calculadora.multiplicacion(5, 3)
print("Multiplicación:", resulMultiplicacion)

# Importar el módulo con un alias
import utils.texto as txt

print("Mayúsculas:", txt.mayusculas("hola"))

# Importar funciones específicas
from utils.calculadora import potencia

print("Potencia:", potencia(10, 2))

# Importar múltiples funciones en una sola línea
from utils.texto import mayusculas, minusculas

print("Minúsculas:", minusculas("HOLA"))

# Importar todo el contenido del módulo (No recomendado en proyectos grandes)
from utils.calculadora import *

print("Suma con *:", suma(2, 2))
```

3.2. Importar contenido en *__init__.py*

El fichero __init__.py que antes de la versión 3.3 de Python era obligatorio para poder realizar la importación de recursos de un módulo a otro (ya no lo es) tiene la particularidad de que con él se pueden definir importaciones a nivel de módulo, actuando como intermediario o enrutador entre los módulos de una carpeta y sus recursos internos.

Veamos el siguiente ejemplo:

Imaginemos la siguiente estructura.

```
zoologico/
├── main.py
├── animales/
│   ├── __init__.py
│   ├── vertebrados.py
│   ├── invertebrados.py
```

vertebrados.py

```
class vertebrado:
    def __init__(self, nombre, tipo):
        self.nombre = nombre
        self.tipo = tipo # Mamífero, Ave, Reptil, etc.
    def mostrar_info(self):
        return f"{self.nombre} es un vertebrado de tipo {self.tipo}."
```

invertebrados.py

```
class invertebrado:
    def __init__(self, nombre, tipo):
        self.nombre = nombre
        self.tipo = tipo # Molusco, artrópodo, etc.
    def mostrar_info(self):
        return f"{self.nombre} es un invertebrado de tipo {self.tipo}."
```

Código en __init__.py

```
from .vertebrados import vertebrado
from .invertebrados import invertebrado
```

Esto va a permitir que **animales/** sea reconocido como un paquete en Python y se pueda importar vertebrado e invertebrado sin necesidad de especificar cada módulo.

Ahora:

Código en main.py

```
from animales import vertebrado, invertebrado
# creamos las instancias
perro = vertebrado("Perro", "Mamífero")
pulpo = invertebrado("Pulpo", "Molusco")
# mostrar información
print(perro.mostrar_info()) # Perro es un vertebrado de tipo Mamífero.
print(pulpo.mostrar_info()) # Pulpo es un invertebrado de tipo Molusco.
```

3.3. Repositorios de paquetes.

La librería estándar de Python cuenta con multitud de paquetes para el desarrollo de aplicaciones. Pero en determinadas situaciones se necesita incorporar paquetes creados por la comunidad o terceros. Estos paquetes se encuentran alojados en **repositorios de paquetes** de los que el más popular es sin duda **Pypi** (https://pypi.org/)

Pero veamos cómo construir paquetes para poder compartir el código…

La información importante de un paquete en Python se encuentra en el fichero ***setup.py*** que debe estar en la raíz del proyecto. Este fichero define las propiedades fundamentales del proyecto como son:

- **name**: nombre del paquete
- **version**: versión del paquete
- **author y author_email**: el nombre y el email del autor
- **description, long_description, long_description_content_type**: para definir información referente a la descripción
- **url**: la dirección del repositorio en la que se puede encontrar el código, normalmente github.com
- **classifiers**: categorías para clasificar el paquete en Pypi
- **python_requires**: define la mínima versión de Python para que se pueda ejecutar el paquete.

Un ejemplo de un fichero **setup.py** básico:

```
from setuptools import setup, find_packages

setup (
    name="mi_paquete",
    version="0.1.0",
    author="Tu Nombre",
    author_email="tuemail@example.com",
    description="Un paquete de ejemplo en Python",
    long_description=open("README.md", encoding="utf-8").read(),
    long_description_content_type="text/markdown",
    url="https://github.com/tuusuario/mi_paquete",
    packages=find_packages(),
    classifiers=[
        "Programming Language :: Python :: 3",
        "License :: OSI Approved :: MIT License",
```

```
        "Operating System :: OS Independent",
    ],
    python_requires='>=3.6',

)
```

4. Paquetes disponibles en la librería estándar.

Es muy importante conocer las herramientas que la librería estándar ofrece para poder afrotar mejor cualquier problema desde Python. Existen una gran cantidad de módulos y paquetes preinstalados. Veremos a continuación un resumen por categorías pero se recomienda encarecidamente revisar cada cierto tiempo la **lista oficial de módulos disponibles en https://docs.python.org/3/library/.**

Manipulación de datos y archivos:

- os→ operaciones del sistema
- shutil→ copia, mueve y elimina archivos y directorios
- pathlib→ manejo de rutas de archivos
- glob→búsqueda de archivos con patrones
- tempfile→ creación de archivos temporales
- csv→ leer y escribir archivos CSV
- json → manejo de datos en formato JSON
- pickle→ serialización y deserialización de objetos Python
- sqlite3→ base de dartos SQLite integrada

Programación Funcional y Estructura de datos:

- collections→ estructuras avanzadas como **deque**, **Counter**, **defaultdict.**
- itertools→ iteradores eficientes para combinaciones, permutaciones, etc
- functools → decoradores y funciones de orden superior

Fechas y tiempos:

- time→ manejo de tiempo y pausas de ejecución (sleep, time())
- datetime→ fechas y horas avanzadas
- calendar→ manejo de calendarios

Cálculo y matemáticas:

- math→ funciones matemáticas básicas
- random→ generación de números aleatorios y aleatorización de listas
- statistics→ cálculo de media, mediana, desviación estándar
- decimal→ precisión exacta en cálculos con decimales.

Programación en red y web:

- urllib→ manejo de urls y descargas
- http.client→ cliente HTTP para peticiones
- socket→ programación de sockets (redes)
- smtplib→ envío de correos vía SMTP
- email→ creación y manipulación de correos electrónicos

Manejo de errores y depuración:

- logging→ registro de eventos y logs
- traceback→ análisis de excepciones y errores
- warnings→ gestión de advertencias en tiempo de ejecución

Multiprocesamiento y concurrencia:

- threading→ manejo de hilos en paralelo
- multiprocessing→ uso de múltiples procesos
- asyncio→ programación asíncrona

Seguridad y cifrado:

- hashlib→ cifrado con SHA, MD5, etc.
- hmac→ algoritmos HMAC para autenticación
- secrets→ generación segura de contraseñas y tokens

Como se puede comprobar, Python ofrece una librería estándar muy amplia y completa. Esto es, como hemos mencionado, sólo una parte.